SDLC Review – Case Studies

Case Study #1: Trouble in Paradise

Small organization but undergoing significant growth over the past few years

IT team increased from 2 to 15 during that time

Primary product is a business scheduling / planning applications for 200+ users and used to track customer information, schedules, coordination, and follow-up information.

Organization has a board of directors (including 3 executives), a Business Operations wing, and an IT Team.

There is a history of friction between the Board and the IT Team, as well as a moderate level of turnover among the Board.

The Board's concerns:

- Director of IT has too much independence
- Worried about being held over a barrel – director of IT could lock out access to systems, infrastructure, code and Board would have no recourse or protection
- Unclear on what IT priorities are, or when urgent requests are going to be delivered
- History of new changes/features being added that they were not aware of, and having uncertain business value

1. **Initiation**
    a. Intake
    b. Scoping
    c. Charter
2. **Requirements**
    a. Authoring
    b. Modification
    c. Clarification
    d. Approval
    e. Traceability
        i. Commits
        ii. Dependencies
        iii. Test Creation / Execution
        iv. Versioning / Delivery
        v. Defects
    f. Acceptance
3. **Planning**
    a. Estimation
    b. Scheduling
    c. Allocations

Director of IT's perspective:

- Serves in the role of Product Owner as well as Director of IT
- Requests are received from many business stakeholders as well as generated by IT and the Product Owner
- Stakeholders can have competing business interests and priorities
- Works items are tracked in JIRA are a linked to commits, test cases, and releases informally and inconsistently by comments and documentation
- Status and Priorities of planned work and schedule is reviewed with the Board at monthly meetings via powerpoint/excel which includes formal signoff and approval of business requests / enhancements

Discuss:

1) What potential risks do you see here
2) Identify two (or more) recommendations to make to the client
3) What do you know about how MNP does this?
4) Suggest one (or more) recommendations to implement at MNP.

Case Study #2: The Times they are a Changin'

A medium-size logistics organization operates with a relatively lean model, with minimal full-time staff who often have multiple responsibilities

Business is highly dependent on data and interoperability which is becoming increasingly central to their business model.

IT team serves as a dual function development & operations team.

Due to business focus on data, most developers have stronger database development skills and experience, and most business logic is implemented in database stored procedures to facilitate sharing and reuse.

Business tools were previously split into dozens of different apps, but access to them has now been centralized through a portal that provides access to a wide spectrum of cloud-based operations and apps to all staff.

Each operation/app is *usually mostly* independent with its own deployment, database, and project / source control repository in Azure DevOps. Although some operations/apps share common information, business rules, or dependencies – those are not currently being tracked.

1. **Source Control**
    a. Access
    b. Commit Policy
    c. Changeset Delivery / Acceptance
    d. Branch/Merge Policy
    e. Gates
2. **Configuration**
    a. Security (Encryption)
    b. Storage
    c. Application Configuration
    d. Environment Configuration
3. **Documentation**
    a. In-Code Documentation
    b. User Documentation
    c. Policy / Procedure / Training Documentation
4. **Continuous Integration**
    a. Build Automation
    b. Test Automation
    c. Deploy Automation
    d. Security Policy
    e. Artifact Management

Each operation/app is independently configured with configuration stored in a secure Azure environment configuration.

Source control does not implement any quality gates or restrictions on branching, merging, commits, or review. All developers have full access to all branches, but only application code is being stored and versioned.

All apps have build and deploy automation configured (which are triggered manually) but are simple one or two-step pipelines (Restore nuget libraries; Compile; Azure Function Deploy).

As part of the standard process, the Development Lead is the only person with build/deploy permissions and performs a code review on each release prior to deployment.

Development processes, standards, and expectations have limited or no documentation. Coding standards are primarily stylistic and simple programmatic standards (naming conventions, basic runtime error prevention) and are documented in a shared word document that provides suggested directives only without additional context. Ie. ("Avoid dynamic SQL statements as much as possible.") Code commenting is recommended, but inconsistently applied.

Discuss:

1) What potential risks do you see here
2) Identify two (or more) recommendations to make to the client
3) What do you know about how MNP does this?
4) Suggest one (or more) recommendations to implement at MNP.

Case Study #3: Quality is Job 1

An organization that serves public-sector and law-enforcement clients with financial tools for b udgeting and expenditure tracking.

Applications are multi-tenant, single code-base and hosted on-premise with a blue/green deployment model, frequent incremental and full database backups and offsite cold storage. The total number of users in the system is relatively limited (fewer than 200) and does not undergo significant fluctuation in the typical number of active users.

Some clients have special case requirements, so although the application is a single-code base – there are a few special rules that are hardcoded to be client-specific that still exist in the application.

Any new feature requests or requirements include specific and detailed list of application changes and acceptance criteria.

Test scripts are stored in a special tool that includes setup steps, test data, test steps and validation criteria, and allow for tracking of test suits, test plans, and test execution tracking and success/failure. Clients are given access to the test management tool and encouraged to review and utilize the tool to aid in User Acceptance testing.

1. **Unit Tests**
    a. Unit Test Policy
    b. Coverage Metrics
    c. Execution Frequency
2. **Functional Tests**
    a. Test Scripts
    b. Test Automation
    c. Coverage Metrics
    d. Environment Smoke-tests
    e. User Acceptance
3. **Non-Functional Tests**
    a. Security (Vulnerability and Penetration Testing)
    b. Scalability
    c. Performance
    d. Reliability / Failover
    e. Load/Volume
4. **Maintenance Tests**
    a. Regression

Standard processes for release planning include creating a test plan that includes any new test scripts required for new features in the release, in addition to any existing test cases directly related to modified features. Tests included in the test plan must be completed and marked successful before a release is approved for production.

All test scripts are manual, there is no test automation or unit testing in place.

Vulnerability scanning is manual and is completed periodically (typically monthly) by the Director of IT. Although the application deals with financial information and budgets, the information is technically accessible to the public via FOIA (freedom of information act) requests so third party independent penetration testing is not a priority.

Discuss:

1) What potential risks do you see here
2) Identify two (or more) recommendations to make to the client
3) What do you know about how MNP does this?
4) Suggest one (or more) recommendations to implement at MNP.